

TypeScript Fundamentals - Part 4

Interfaces & Classes

Understanding Interfaces in TypeScript

Interfaces are one of TypeScript's most powerful features for defining contracts in your code. An interface defines the shape of an object, specifying what properties and methods it should have, along with their types.

What are Interfaces?

An interface is a structure that defines the contract in your application. It defines the syntax for classes to follow, meaning a class that implements an interface must implement all the properties and methods defined in that interface.

Think of an interface as a blueprint. Just as a building blueprint specifies what rooms a house should have, an interface specifies what properties and methods an object should have.

Defining Interfaces

You define an interface using the `interface` keyword followed by the interface name and its structure.

```
// Basic interface definition
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// Using the interface
const user1: User = {
  id: 1,
  name: "Rajesh Kumar",
  email: "rajesh@example.com",
  age: 28
};
```

```
const user2: User = {
  id: 2,
  name: "Priya Sharma",
  email: "priya@example.com",
  age: 25
};

console.log("User 1:", user1);
console.log("User 2:", user2);

// Function accepting interface type
function displayUser(user: User): void {
  console.log(`ID: ${user.id}, Name: ${user.name}, Email: ${user.email}, Age: ${user.age}`);
}

displayUser(user1);
displayUser(user2);
```

Output:

```
User 1: { id: 1, name: 'Rajesh Kumar', email: 'rajesh@example.com', age: 28 }
User 2: { id: 2, name: 'Priya Sharma', email: 'priya@example.com', age: 25 }
ID: 1, Name: Rajesh Kumar, Email: rajesh@example.com, Age: 28
ID: 2, Name: Priya Sharma, Email: priya@example.com, Age: 25
```

When you define an object with an interface type, TypeScript ensures that the object has all the required properties with the correct types. If you miss a property or use the wrong type, TypeScript will show an error.

```
// This will cause errors:
// const invalidUser: User = {
//   id: 3,
//   name: "Invalid"
//   // ✖ Missing 'email' and 'age' properties
// };

// const wrongType: User = {
//   id: "4", // ✖ Error: Type 'string' is not assignable to type 'number'
//   name: "Wrong",
```

```
// email: "wrong@example.com",  
// age: 30  
// };
```

Optional Properties in Interfaces

Sometimes, not all properties of an interface are required. You can mark a property as optional by adding a question mark (?) after the property name.

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  description?: string; // Optional property  
  discount?: number;    // Optional property  
}  
  
// Valid: includes optional properties  
const product1: Product = {  
  id: 101,  
  name: "Laptop",  
  price: 50000,  
  description: "High-performance laptop",  
  discount: 10  
};  
  
// Also valid: omits optional properties  
const product2: Product = {  
  id: 102,  
  name: "Mouse",  
  price: 500  
};  
  
console.log("Product 1:", product1);  
console.log("Product 2:", product2);  
  
function displayProduct(product: Product): void {  
  console.log(`${product.name} - ₹${product.price}`);  
  if (product.description) {  
    console.log(`Description: ${product.description}`);  
  }  
  if (product.discount) {  
    console.log(`Discount: ${product.discount}%`);  
  }  
}
```

```
}  
  
displayProduct(product1);  
console.log("---");  
displayProduct(product2);
```

Output:

```
Product 1: { id: 101, name: 'Laptop', price: 50000, description: 'High-performance laptop',  
discount: 10 }  
Product 2: { id: 102, name: 'Mouse', price: 500 }  
Laptop - ₹50000  
Description: High-performance laptop  
Discount: 10%  
---  
Mouse - ₹500
```

Readonly Properties

The `readonly` keyword makes a property immutable after the object is created. You can assign a value during initialization, but you cannot modify it afterwards.

```
interface Book {  
    readonly isbn: string;  
    readonly title: string;  
    author: string;  
    pages: number;  
}  
  
const book: Book = {  
    isbn: "978-0-123456-78-9",  
    title: "TypeScript Fundamentals",  
    author: "John Doe",  
    pages: 350  
};  
  
console.log("Book:", book);  
  
// You can modify non-readonly properties
```

```
book.author = "Jane Smith";
book.pages = 375;
console.log("Updated author and pages:", book);

// But you cannot modify readonly properties:
// book.isbn = "978-0-987654-32-1"; // ✗ Error: Cannot assign to 'isbn' because it is a read-
only property
// book.title = "New Title"; // ✗ Error: Cannot assign to 'title' because it is a read-only
property
```

Output:

```
Book: { isbn: '978-0-123456-78-9', title: 'TypeScript Fundamentals', author: 'John Doe', pages:
350 }
Updated author and pages: { isbn: '978-0-123456-78-9', title: 'TypeScript Fundamentals',
author: 'Jane Smith', pages: 375 }
```

Interface for Function Types

Interfaces can also describe function types. You specify the parameter types and return type in the interface.

```
// Interface for a function
interface MathOperation {
  (a: number, b: number): number;
}

// Implementing the interface
const add: MathOperation = (x, y) => x + y;
const multiply: MathOperation = (x, y) => x * y;
const subtract: MathOperation = (x, y) => x - y;

console.log("Add:", add(10, 5));
console.log("Multiply:", multiply(10, 5));
console.log("Subtract:", subtract(10, 5));

// Another example: string processor
interface StringProcessor {
  (input: string): string;
}
```

```
const toUpperCase: StringProcessor = (str) => str.toUpperCase();
const toLowerCase: StringProcessor = (str) => str.toLowerCase();
const reverse: StringProcessor = (str) => str.split('').reverse().join('');

console.log("Uppercase:", toUpperCase("hello"));
console.log("Lowercase:", toLowerCase("WORLD"));
console.log("Reverse:", reverse("TypeScript"));
```

Output:

```
Add: 15
Multiply: 50
Subtract: 5
Uppercase: HELLO
Lowercase: world
Reverse: tpircSepyT
```

Interface Inheritance (Extending Interfaces)

Interfaces can extend other interfaces, allowing you to create new interfaces based on existing ones. This promotes code reuse and creates hierarchies of related types.

```
// Base interface
interface Person {
  name: string;
  age: number;
}

// Extended interface
interface Employee extends Person {
  employeeId: number;
  department: string;
  salary: number;
}

// Another extended interface
interface Student extends Person {
  studentId: number;
  course: string;
  semester: number;
}
```

```
}

const employee: Employee = {
  name: "Amit Patel",
  age: 30,
  employeeId: 1001,
  department: "Engineering",
  salary: 80000
};

const student: Student = {
  name: "Neha Singh",
  age: 21,
  studentId: 2001,
  course: "Computer Science",
  semester: 6
};

console.log("Employee:", employee);
console.log("Student:", student);

// Interface extending multiple interfaces
interface Address {
  street: string;
  city: string;
  pincode: string;
}

interface Contact {
  phone: string;
  email: string;
}

interface CustomerInfo extends Person, Address, Contact {
  customerId: number;
}

const customer: CustomerInfo = {
  name: "Ravi Kumar",
  age: 35,
  street: "MG Road",
  city: "Bangalore",
  pincode: "560001",
  phone: "9876543210",
  email: "ravi@example.com",
  customerId: 5001
}
```

```
};

console.log("Customer:", customer);
```

Output:

```
Employee: { name: 'Amit Patel', age: 30, employeeId: 1001, department: 'Engineering', salary:
80000 }
Student: { name: 'Neha Singh', age: 21, studentId: 2001, course: 'Computer Science', semester:
6 }
Customer: { name: 'Ravi Kumar', age: 35, street: 'MG Road', city: 'Bangalore', pincode:
'560001', phone: '9876543210', email: 'ravi@example.com', customerId: 5001 }
```

Interfaces vs Type Aliases

Both interfaces and type aliases can be used to define object shapes, but there are some differences between them.

Feature	Interface	Type Alias
Syntax	<code>interface Name { }</code>	<code>type Name = { }</code>
Extension	Uses <code>extends</code> keyword	Uses intersection <code>&</code>
Declaration Merging	Supported (can declare same interface multiple times)	Not supported
Union Types	Cannot represent unions directly	Can represent unions
Primitives	Cannot alias primitives	Can alias primitives
Use Case	Object shapes, class contracts	Complex types, unions, primitives

```
// Interface example
interface Point {
  x: number;
```



```
y: number;
}

// Type alias example (functionally similar)
type PointType = {
  x: number;
  y: number;
};

const p1: Point = { x: 10, y: 20 };
const p2: PointType = { x: 30, y: 40 };

console.log("Point with interface:", p1);
console.log("Point with type:", p2);

// Type aliases can do things interfaces cannot:
type ID = string | number; // Union type
type Status = "active" | "inactive" | "pending"; // Literal union

let userId: ID = "USER123";
let userStatus: Status = "active";

console.log("User ID:", userId);
console.log("Status:", userStatus);
```

Output:

```
Point with interface: { x: 10, y: 20 }
Point with type: { x: 30, y: 40 }
User ID: USER123
Status: active
```

When to Use Interfaces

Interfaces are best suited for:

- Defining the shape of objects
- Creating contracts that classes must follow
- Working with object-oriented programming patterns
- When you need declaration merging

- Public API definitions in libraries

Tip:

Best Practice: Use interfaces for object shapes and when defining contracts for classes. Use type aliases for unions, intersections, mapped types, and when you need more complex type operations.

Classes in TypeScript

Understanding Classes

Classes are blueprints for creating objects. They encapsulate data (properties) and behavior (methods) into a single unit. TypeScript enhances JavaScript classes with type annotations and additional features like access modifiers.

What is a Class?

A class is a template for creating objects. It defines what properties an object will have and what actions (methods) it can perform. Think of a class as a cookie cutter and objects as the cookies it creates.

What is an Object?

An object is an instance of a class. When you create an object from a class, you're creating a specific example of that class with its own property values.

Class Syntax in TypeScript

Here's the basic syntax for defining and using a class:

```
// Class definition
class Person {
  // Properties (member variables)
  name: string;
  age: number;
  city: string;
```

```
// Constructor method
constructor(name: string, age: number, city: string) {
  this.name = name;
  this.age = age;
  this.city = city;
}

// Method
introduce(): void {
  console.log(`Hello, I am ${this.name}, ${this.age} years old, from ${this.city}.`);
}

// Creating objects (instances) of the class
const person1 = new Person("Rajesh", 28, "Mumbai");
const person2 = new Person("Priya", 25, "Delhi");

console.log("Person 1:", person1);
console.log("Person 2:", person2);

person1.introduce();
person2.introduce();
```

Output:

```
Person 1: Person { name: 'Rajesh', age: 28, city: 'Mumbai' }
Person 2: Person { name: 'Priya', age: 25, city: 'Delhi' }
Hello, I am Rajesh, 28 years old, from Mumbai.
Hello, I am Priya, 25 years old, from Delhi.
```

Constructor Method

The constructor is a special method that runs when you create a new instance of a class. It's used to initialize the object's properties.

```
class Car {
  brand: string;
  model: string;
  year: number;
  price: number;
```

```
constructor(brand: string, model: string, year: number, price: number) {
    this.brand = brand;
    this.model = model;
    this.year = year;
    this.price = price;
    console.log(`New car created: ${brand} ${model}`);
}

displayInfo(): void {
    console.log(`${this.year} ${this.brand} ${this.model} - ₹${this.price}`);
}

applyDiscount(percentage: number): void {
    this.price = this.price - (this.price * percentage / 100);
    console.log(`Discount applied! New price: ₹${this.price}`);
}
}

const car1 = new Car("Toyota", "Fortuner", 2024, 3500000);
const car2 = new Car("Honda", "City", 2024, 1200000);

car1.displayInfo();
car2.displayInfo();

car1.applyDiscount(10);
car1.displayInfo();
```

Output:

```
New car created: Toyota Fortuner
New car created: Honda City
2024 Toyota Fortuner - ₹3500000
2024 Honda City - ₹1200000
Discount applied! New price: ₹3150000
2024 Toyota Fortuner - ₹3150000
```

Properties and Methods

Properties store data about the object, while methods define what the object can do.

```
class BankAccount {
  accountNumber: string;
  accountHolder: string;
  balance: number;

  constructor(accountNumber: string, accountHolder: string, initialBalance: number) {
    this.accountNumber = accountNumber;
    this.accountHolder = accountHolder;
    this.balance = initialBalance;
  }

  deposit(amount: number): void {
    if (amount > 0) {
      this.balance += amount;
      console.log(`Deposited ₹${amount}. New balance: ₹${this.balance}`);
    } else {
      console.log("Invalid deposit amount");
    }
  }

  withdraw(amount: number): void {
    if (amount > 0 && amount <= this.balance) {
      this.balance -= amount;
      console.log(`Withdrawn ₹${amount}. New balance: ₹${this.balance}`);
    } else {
      console.log("Invalid withdrawal amount or insufficient funds");
    }
  }

  getBalance(): number {
    return this.balance;
  }

  displayAccountInfo(): void {
    console.log(`Account: ${this.accountNumber}`);
    console.log(`Holder: ${this.accountHolder}`);
    console.log(`Balance: ₹${this.balance}`);
  }
}

const account = new BankAccount("ACC001", "Amit Kumar", 10000);
account.displayAccountInfo();
console.log("---");

account.deposit(5000);
account.withdraw(3000);
```

```
account.withdraw(20000);  
console.log("---");  
  
account.displayAccountInfo();
```

Output:

```
Account: ACC001  
Holder: Amit Kumar  
Balance: ₹10000  
---  
Deposited ₹5000. New balance: ₹15000  
Withdrawn ₹3000. New balance: ₹12000  
Invalid withdrawal amount or insufficient funds  
---  
Account: ACC001  
Holder: Amit Kumar  
Balance: ₹12000
```

Access Modifiers in TypeScript

Access modifiers control the visibility and accessibility of class members (properties and methods). TypeScript provides three access modifiers: `public`, `private`, and `protected`.

Public Access Modifier

The `public` modifier makes a property or method accessible from anywhere. This is the default modifier in TypeScript, so if you don't specify any modifier, the member is public.

```
class Employee {  
    public name: string; // Explicitly public  
    public employeeId: number;  
    department: string; // Implicitly public (no modifier specified)  
  
    constructor(name: string, employeeId: number, department: string) {  
        this.name = name;  
        this.employeeId = employeeId;  
    }  
}
```

```
        this.department = department;
    }

    public displayInfo(): void {
        console.log(`${this.name} (ID: ${this.employeeId}) - ${this.department}`);
    }
}

const emp = new Employee("Ravi Sharma", 101, "IT");

// All public members are accessible
console.log("Name:", emp.name);
console.log("Employee ID:", emp.employeeId);
console.log("Department:", emp.department);
emp.displayInfo();
```

Output:

```
Name: Ravi Sharma
Employee ID: 101
Department: IT
Ravi Sharma (ID: 101) - IT
```

Private Access Modifier

The `private` modifier makes a property or method accessible only within the class itself. It cannot be accessed from outside the class or from derived classes.

```
class BankAccountSecure {
    public accountNumber: string;
    public accountHolder: string;
    private balance: number; // Private property
    private pin: string;     // Private property

    constructor(accountNumber: string, accountHolder: string, initialBalance: number, pin:
string) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.pin = pin;
    }
}
```

```
// Public method to access private balance
public getBalance(enteredPin: string): number | string {
    if (this.verifyPin(enteredPin)) {
        return this.balance;
    }
    return "Invalid PIN";
}

// Private method
private verifyPin(enteredPin: string): boolean {
    return this.pin === enteredPin;
}

public deposit(amount: number, enteredPin: string): void {
    if (!this.verifyPin(enteredPin)) {
        console.log("Invalid PIN. Transaction failed.");
        return;
    }
    if (amount > 0) {
        this.balance += amount;
        console.log(`Deposited ₹${amount}. New balance: ₹${this.balance}`);
    }
}

public withdraw(amount: number, enteredPin: string): void {
    if (!this.verifyPin(enteredPin)) {
        console.log("Invalid PIN. Transaction failed.");
        return;
    }
    if (amount > 0 && amount <= this.balance) {
        this.balance -= amount;
        console.log(`Withdrawn ₹${amount}. New balance: ₹${this.balance}`);
    } else {
        console.log("Invalid amount or insufficient funds");
    }
}
}

const secureAccount = new BankAccountSecure("ACC123", "Neha Patel", 50000, "1234");

// Can access public properties
console.log("Account Number:", secureAccount.accountNumber);
console.log("Account Holder:", secureAccount.accountHolder);

// Cannot access private properties directly:
// console.log(secureAccount.balance); // ✗ Error
```



```
// console.log(secureAccount.pin); // ✗ Error

// Must use public methods with PIN
console.log("Balance:", secureAccount.getBalance("1234"));
console.log("Balance with wrong PIN:", secureAccount.getBalance("0000"));

secureAccount.deposit(10000, "1234");
secureAccount.withdraw(5000, "1234");
secureAccount.withdraw(5000, "0000");
```

Output:

```
Account Number: ACC123
Account Holder: Neha Patel
Balance: 50000
Balance with wrong PIN: Invalid PIN
Deposited ₹10000. New balance: ₹60000
Withdrawn ₹5000. New balance: ₹55000
Invalid PIN. Transaction failed.
```

Protected Access Modifier

The `protected` modifier makes a property or method accessible within the class and its subclasses (derived classes), but not from outside.

```
class Vehicle {
    protected brand: string;
    protected model: string;
    private engineNumber: string;

    constructor(brand: string, model: string, engineNumber: string) {
        this.brand = brand;
        this.model = model;
        this.engineNumber = engineNumber;
    }

    protected displayBasicInfo(): void {
        console.log(`${this.brand} ${this.model}`);
    }
}
```

```
public getInfo(): void {
    this.displayBasicInfo();
    console.log(`Engine: ${this.engineNumber}`);
}
}

class Motorcycle extends Vehicle {
    private type: string;

    constructor(brand: string, model: string, engineNumber: string, type: string) {
        super(brand, model, engineNumber);
        this.type = type;
    }

    public displayMotorcycleInfo(): void {
        // Can access protected members from parent class
        console.log(`Motorcycle: ${this.brand} ${this.model}`);
        console.log(`Type: ${this.type}`);
        this.displayBasicInfo(); // Can call protected method

        // Cannot access private members:
        // console.log(this.engineNumber); // ✗ Error
    }
}

const bike = new Motorcycle("Royal Enfield", "Classic 350", "ENG12345", "Cruiser");

bike.getInfo();
console.log("---");
bike.displayMotorcycleInfo();

// Cannot access protected members from outside:
// console.log(bike.brand); // ✗ Error
// bike.displayBasicInfo(); // ✗ Error
```

Output:

```
Royal Enfield Classic 350
```

```
Engine: ENG12345
```

```
---
```

Motorcycle: Royal Enfield Classic 350

Type: Cruiser

Royal Enfield Classic 350

Readonly Properties in Classes

Readonly properties can only be assigned during initialization (in the property declaration or in the constructor). After that, they cannot be modified.

```
class Product {
  readonly productId: string;
  readonly createdAt: Date;
  public name: string;
  public price: number;

  constructor(productId: string, name: string, price: number) {
    this.productId = productId;
    this.createdAt = new Date();
    this.name = name;
    this.price = price;
  }

  updatePrice(newPrice: number): void {
    this.price = newPrice;
    console.log(`Price updated to ₹${newPrice}`);
  }

  displayProduct(): void {
    console.log(`ID: ${this.productId}`);
    console.log(`Name: ${this.name}`);
    console.log(`Price: ₹${this.price}`);
    console.log(`Created: ${this.createdAt.toLocaleDateString()}`);
  }
}

const product = new Product("PROD001", "Laptop", 50000);
product.displayProduct();

// Can modify non-readonly properties
product.name = "Gaming Laptop";
product.updatePrice(55000);
product.displayProduct();
```

```
// Cannot modify readonly properties:
// product.productId = "PROD002"; // ✗ Error
// product.createdDate = new Date(); // ✗ Error
```

Output:

```
ID: PROD001
Name: Laptop
Price: ₹50000
Created: 1/21/2025
Price updated to ₹55000
ID: PROD001
Name: Gaming Laptop
Price: ₹55000
Created: 1/21/2025
```

When to Use Each Access Modifier

Modifier	Accessible From	Use Case
public	Anywhere	Data and methods that should be available to all code
private	Within the class only	Internal implementation details, sensitive data
protected	Within class and subclasses	Data shared with derived classes but hidden from outside
readonly	Based on other modifiers	Data that should never change after initialization

Object-Oriented Programming Concepts in TypeScript

Property Methods and Constructor Functions

Let's explore how TypeScript enhances object-oriented programming with better type safety and additional features.

Constructor Function in Detail

TypeScript allows you to use parameter properties, which is a shorthand for declaring and initializing properties directly in the constructor.

```
// Traditional approach
class UserTraditional {
  public name: string;
  public email: string;
  private age: number;

  constructor(name: string, email: string, age: number) {
    this.name = name;
    this.email = email;
    this.age = age;
  }
}

// Shorthand approach using parameter properties
class UserShorthand {
  constructor(
    public name: string,
    public email: string,
    private age: number
  ) {
    // Properties are automatically created and initialized
  }

  getAge(): number {
    return this.age;
  }
}

const user = new UserShorthand("Vikram", "vikram@example.com", 30);
console.log("Name:", user.name);
console.log("Email:", user.email);
```

```
console.log("Age:", user.getAge());

// Another example with methods
class Rectangle {
  constructor(
    private width: number,
    private height: number
  ) {
    console.log(`Rectangle created: ${width} x ${height}`);
  }

  calculateArea(): number {
    return this.width * this.height;
  }

  calculatePerimeter(): number {
    return 2 * (this.width + this.height);
  }

  resize(newWidth: number, newHeight: number): void {
    this.width = newWidth;
    this.height = newHeight;
    console.log(`Resized to: ${newWidth} x ${newHeight}`);
  }

  display(): void {
    console.log(`Width: ${this.width}, Height: ${this.height}`);
    console.log(`Area: ${this.calculateArea()}`);
    console.log(`Perimeter: ${this.calculatePerimeter()}`);
  }
}

const rect = new Rectangle(10, 5);
rect.display();
console.log("---");

rect.resize(15, 8);
rect.display();
```

Output:

```
Rectangle created: 10 x 5
Name: Vikram
Email: vikram@example.com
```

```
Age: 30
Width: 10, Height: 5
Area: 50
Perimeter: 30
---
Resized to: 15 x 8
Width: 15, Height: 8
Area: 120
Perimeter: 46
```

Method Definitions and Overloading

Methods can be overloaded to provide different signatures, and you can define both instance methods and static methods.

```
class Calculator {
  // Method overloading
  add(a: number, b: number): number;
  add(a: string, b: string): string;
  add(a: number[], b: number[]): number[];

  // Implementation
  add(a: any, b: any): any {
    if (typeof a === "number" && typeof b === "number") {
      return a + b;
    } else if (typeof a === "string" && typeof b === "string") {
      return a + b;
    } else if (Array.isArray(a) && Array.isArray(b)) {
      return [...a, ...b];
    }
  }

  // Static method - can be called without creating an instance
  static multiply(a: number, b: number): number {
    return a * b;
  }

  // Instance method
  power(base: number, exponent: number): number {
    return Math.pow(base, exponent);
  }
}
```

```
}  
  
const calc = new Calculator();  
  
console.log("Add numbers:", calc.add(5, 10));  
console.log("Add strings:", calc.add("Hello ", "World"));  
console.log("Add arrays:", calc.add([1, 2], [3, 4]));  
console.log("Power:", calc.power(2, 3));  
  
// Static method - called on the class itself  
console.log("Static multiply:", Calculator.multiply(6, 7));
```

Output:

```
Add numbers: 15  
Add strings: Hello World  
Add arrays: [ 1, 2, 3, 4 ]  
Power: 8  
Static multiply: 42
```

Converting JavaScript Functions to TypeScript Classes with OOP

Let's see how to convert traditional JavaScript function-based code into TypeScript classes with proper OOP principles.

JavaScript Function Approach

```
// JavaScript-style function  
function createStudent(name, rollNumber, marks) {  
  return {  
    name: name,  
    rollNumber: rollNumber,  
    marks: marks,  
    getGrade: function() {  
      if (this.marks >= 90) return 'A+';  
      if (this.marks >= 80) return 'A';  
      if (this.marks >= 70) return 'B';  
      if (this.marks >= 60) return 'C';  
      return 'F';  
    },  
  },  
}
```



```
        display: function() {
            console.log(`${this.name} (${this.rollNumber}): ${this.marks} marks - Grade $
{this.getGrade()}`);
        }
    };
}

const student1 = createStudent("Rahul", "S001", 85);
student1.display();
```

Output:

```
Rahul (S001): 85 marks - Grade A
```

TypeScript Class Approach with OOP

```
// TypeScript class with OOP principles
class Student {
    private name: string;
    private rollNumber: string;
    private marks: number;

    constructor(name: string, rollNumber: string, marks: number) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.marks = marks;
    }

    // Getter method
    public getName(): string {
        return this.name;
    }

    public getRollNumber(): string {
        return this.rollNumber;
    }

    public getMarks(): number {
        return this.marks;
    }

    // Setter method with validation
    public setMarks(marks: number): void {
```

```
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        } else {
            console.log("Invalid marks. Must be between 0 and 100.");
        }
    }

    // Business logic method
    public getGrade(): string {
        if (this.marks >= 90) return 'A+';
        if (this.marks >= 80) return 'A';
        if (this.marks >= 70) return 'B';
        if (this.marks >= 60) return 'C';
        return 'F';
    }

    // Display method
    public display(): void {
        console.log(`${this.name} (${this.rollNumber}): ${this.marks} marks - Grade $
{this.getGrade()}`);
    }
}

const student2 = new Student("Priya", "S002", 92);
student2.display();

student2.setMarks(78);
student2.display();

student2.setMarks(150); // Invalid marks
student2.display();
```

Output:

```
Priya (S002): 92 marks - Grade A+
Priya (S002): 78 marks - Grade B
Invalid marks. Must be between 0 and 100.
Priya (S002): 78 marks - Grade B
```

Comprehensive Example: E-commerce System

Let's build a complete example that demonstrates all the OOP concepts we've learned.

```
// Interface for products
interface IProduct {
    id: string;
    name: string;
    price: number;
    stock: number;
}

// Base Product class
class Product implements IProduct {
    constructor(
        public readonly id: string,
        public name: string,
        public price: number,
        private stock: number
    ) {}

    public getStock(): number {
        return this.stock;
    }

    public updateStock(quantity: number): void {
        this.stock = quantity;
    }

    public isAvailable(): boolean {
        return this.stock > 0;
    }

    public display(): void {
        console.log(`${this.name} - ₹${this.price} (Stock: ${this.stock})`);
    }
}

// Shopping Cart class
class ShoppingCart {
    private items: Map;

    constructor() {
        this.items = new Map();
    }

    public addItem(product: Product, quantity: number): void {
        if (!product.isAvailable()) {
            console.log(`${product.name} is out of stock`);
            return;
        }
    }
}
```

```
}

if (quantity > product.getStock()) {
  console.log(`Only ${product.getStock()} units available`);
  return;
}

if (this.items.has(product.id)) {
  const existing = this.items.get(product.id)!;
  existing.quantity += quantity;
  console.log(`Updated ${product.name} quantity to ${existing.quantity}`);
} else {
  this.items.set(product.id, { product, quantity });
  console.log(`Added ${quantity} x ${product.name} to cart`);
}
}

public removeItem(productId: string): void {
  if (this.items.delete(productId)) {
    console.log("Item removed from cart");
  } else {
    console.log("Item not found in cart");
  }
}

public calculateTotal(): number {
  let total = 0;
  this.items.forEach(item => {
    total += item.product.price * item.quantity;
  });
  return total;
}

public displayCart(): void {
  console.log("=== Shopping Cart ===");
  if (this.items.size === 0) {
    console.log("Cart is empty");
    return;
  }

  this.items.forEach(item => {
    console.log(`${item.product.name} x ${item.quantity} = ₹${item.product.price *
item.quantity}`);
  });
  console.log(`Total: ₹${this.calculateTotal()}`);
  console.log("=====");
}
```

```
}

// Create products
const laptop = new Product("P001", "Laptop", 50000, 5);
const mouse = new Product("P002", "Mouse", 500, 10);
const keyboard = new Product("P003", "Keyboard", 1500, 0);

// Create shopping cart
const cart = new ShoppingCart();

// Add items to cart
cart.addItem(laptop, 1);
cart.addItem(mouse, 2);
cart.addItem(keyboard, 1); // Out of stock

cart.displayCart();

// Add more of the same item
cart.addItem(mouse, 3);
cart.displayCart();

// Remove an item
cart.removeItem("P001");
cart.displayCart();
```

Output:

```
Added 1 x Laptop to cart
Added 2 x Mouse to cart
Keyboard is out of stock
=== Shopping Cart ===
Laptop x 1 = ₹50000
Mouse x 2 = ₹1000
Total: ₹51000
=====
Updated Mouse quantity to 5
=== Shopping Cart ===
Laptop x 1 = ₹50000
Mouse x 5 = ₹2500
Total: ₹52500
```

```
=====
Item removed from cart
=== Shopping Cart ===
Mouse x 5 = ₹2500
Total: ₹2500
=====
```

Summary

In this section, we explored interfaces and classes, the cornerstones of object-oriented programming in TypeScript. You learned how interfaces define contracts for object shapes and how they can be extended to create hierarchies. We covered the differences between interfaces and type aliases, helping you understand when to use each.

We dove deep into classes, learning about constructors, properties, methods, and how to create instances. You mastered access modifiers (public, private, protected) and readonly properties, understanding how they control encapsulation and data protection. We also explored how to convert JavaScript function-based code into proper TypeScript classes with OOP principles.

Key takeaways: use interfaces to define object contracts, leverage access modifiers to protect sensitive data, use readonly for immutable properties, and apply OOP principles to create well-structured, maintainable code. TypeScript's class system builds on JavaScript's classes by adding type safety and additional features that make your code more robust and easier to maintain.

With these fundamentals mastered, you now have a solid foundation in TypeScript. You can create type-safe applications using modern JavaScript features enhanced with TypeScript's powerful type system and object-oriented capabilities.